

MoDIN Tutorial and Technical Report

Accelerating Message-Driven Computation with the TinyODIN SNN

Laura Kallem¹, Farzad Fatollahi-Fard², Patricia Gonzalez-Guerrero²

¹Diablo Valley College, ²Lawrence Berkeley National Laboratory

Community College Internship (CCI) Summer 2025

Abstract

Artificial intelligence and machine learning applications demand ever-greater computational power, and semiconducting materials are reaching the peak of their physical potential. As a result, conventional processors are hitting critical limitations such as the AI bottleneck and the eventual end of Moore's Law. Innovative computing approaches such as event-based, message driven, and neuromorphic computing are being researched as they could address these limitations and the need for low power and low latency computing. With these solutions, come increased heterogeneity—specialized processors exist together on a single component to increase performance and efficiency. **The Modular system for Acceleration Integration (MoSAIC) is a message-passing heterogeneous computer architecture that allows for the cost effective testing of hardware accelerators and their interactions.** Neuromorphic processors, inspired by biological neural networks, utilize spiking neural networks (SNNs) that communicate through discrete events rather than continuous computation, enabling ultra-low power consumption and massive parallelism compared to traditional computer architectures. Existing neuromorphic architectures, such as ODIN, have achieved high accuracy at incredibly low amounts of energy consumption in tasks like image classification. Implementing the TinyODIN SNN, a simplified version of the ODIN chip, in MoSAIC would allow for the low cost testing of neuromorphic architectures. **MoDIN is the implementation of TinyODIN SNN as a hardware accelerator tile in MoSAIC*.** As a result of the MoDIN project MoSAIC has neuromorphic capabilities and future testing of the TinyODIN SNN and its interaction with other processors can be done through this platform. Using message-driven computation, MoSAIC enables the fast and accurate exploration of new architectures, which is essential for research on post Moore's Law computing.

***Details for using the MoDIN accelerator tile are described in the tutorial on page 4.**

Introduction

As semiconducting materials reach their physical limits, signaling the end of Moore’s Law [1], hardware architectures become increasingly heterogenous and unconventional computing solutions are being explored. MoSAIC is a platform that allows for the testing of heterogeneous architectures on an FPGA, and the potential of neuromorphic architectures may be explored through the implementation of open source SNNs as hardware accelerators, an area which has suffered from a lack of suitable testing environments.

What separates SNNs from other AI processors is their use of brain-inspired learning rules like spike-driven synaptic plasticity (SDSP) [2], which enable reduced chip space and power by moving away from the expensive computations needed by traditional methods. ODIN [3] is one such processor that implements this learning rule.

The motivation of this project is to explore neuromorphic architectures in MoSAIC by testing the interactions that exist between accelerators on an FPGA. The first SNN processor implemented on MoSAIC is TinyODIN, a stripped-down version of ODIN without on-chip learning capabilities. TinyODIN connects 256 artificial (digital) neurons to each other using 64,000 synapses on a crossbar array architecture, allowing fast and efficient neuron intercommunication. TinyODIN uses the leaky integrate and fire (LIF) model, which mimics basic brain cell behavior by accumulating electrical signals (hence the word “integrate”) until they reach a certain threshold, and then “firing” a spike. A modified version of TinyODIN exists as the hardware accelerator MoDIN in the MoSAIC repository.

Progress

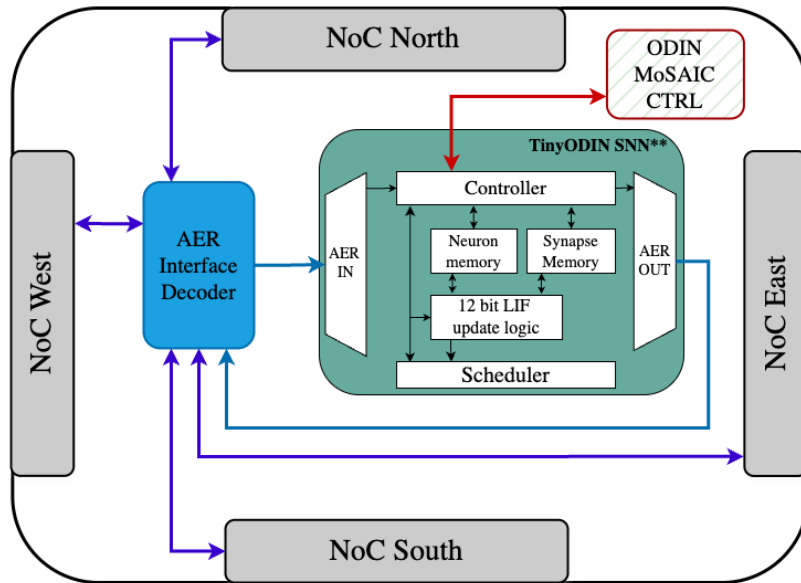


Figure 1: MoDIN accelerator block diagram

TinyODIN Modifications to Create the MoDIN Accelerator

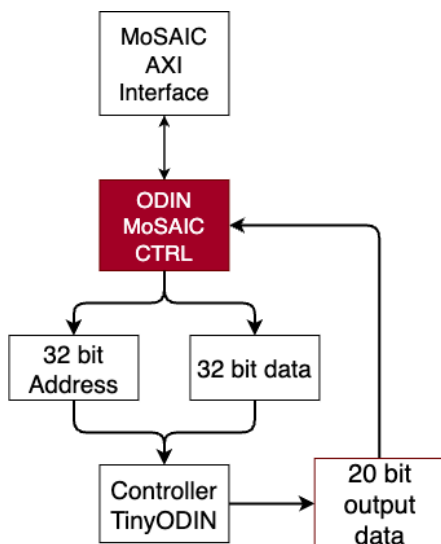


Figure 2: AXI interface flowchart

MoSAIC’s Network on Chip (NoC) system can then be connected to the Address Event Representation (AER) interface in TinyODIN. The AER interface is a neuron spike communication system that was connected to MoSAIC’s NoC network. Spike addresses are taken from the NoC memory address signal, and spike events are triggered when the memory is written to. An AER Interface Decoder was added to translate between MoSAIC’s message format and TinyODIN’s spike format, allowing the neuromorphic processor to communicate with other components in the MoSAIC system while maintaining its event-driven operation.

After TinyODIN is made compatible with MoSAIC, firmware is written that would allow for the initialization of training data and for TinyODIN to run on MoSAIC’s Pico tiles: the system that hosts each hardware accelerator. C code for the Pico allows the user to send messages to MoDIN using the `mPut()` NoC function, and get messages that MoDIN writes to a scratchpad with the `mGet()` NoC function.

Future Work

The implementation of TinyODIN is finished. The biggest objective of future work on this project would be implementing features from the ODIN SNN, giving MoDIN on chip learning capabilities that would make training significantly easier.

Instead of TinyODIN’s original signal configuration interface, the Serial Peripheral (SPI) Interface, MoDIN uses the MoSAIC AXI Interface through the ODIN MoSAIC CTRL block for set up operations like neuron programming.

Removing the SPI interface was accomplished by rerouting several signals that TinyODIN uses internally from the SPI to ODIN MoSAIC CTRL. This was done in compliance with the system originally created for TinyODIN, where a 20 bit address field sent to the SPI interface allowed the SPI to assign values to several internal TinyODIN signals. The top level verilog file of TinyODIN was modified to reflect this change but internal functionality remains the same.

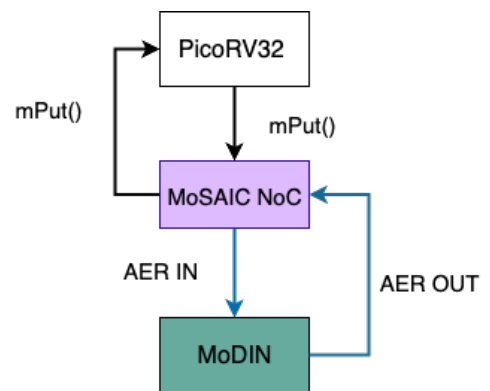


Figure 3: NoC/AER interface flowchart

Verilog File Changes `src/Tile_HDL/modin_tile`

The ODIN_MOSAIC_CTRL block was instantiated in the top level file (now called `tinyMODIN.v`).

`spislave.v` → `ODIN_MOSAIC_CTRL.sv`

The SPI slave block was removed and signals needed for TinyODIN functionality (such as `CTRL_SPI_ADDR`) were translated. New signals `ODINAddr`, `ODIN_wdata`, and `ODIN_data_out_mux` and equivalents to previously existing parameters were added for this purpose.

Previous `spislave.v` logic

```
//SPI_GATE_ACTIVITY - 1 bit - address 0
always @(posedge SCK)
|   if (!spi_addr[17] && !spi_addr[16] && (spi_addr[15:0] == 16'd0 ) && (spi_cnt == 6'd39))
|       SPI_GATE_ACTIVITY <= MOSI;

//SPI_OPEN_LOOP - 1 bit - address 1
always @(posedge SCK)
|   if (!spi_addr[17] && !spi_addr[16] && (spi_addr[15:0] == 16'd1 ) && (spi_cnt == 6'd39))
|       SPI_OPEN_LOOP <= MOSI;

//SPI_AER_SRC_CTRL_nNEUR - 1 bit - address 2
always @(posedge SCK)
|   if (!spi_addr[17] && !spi_addr[16] && (spi_addr[15:0] == 16'd2 ) && (spi_cnt == 6'd39))
|       SPI_AER_SRC_CTRL_nNEUR <= MOSI;

//SPI_MAX_NEUR - M bits - address 3
always @(posedge SCK)
|   if (!spi_addr[17] && !spi_addr[16] && (spi_addr[15:0] == 16'd3 ) && (spi_cnt == 6'd39))
|       SPI_MAX_NEUR <= {spi_shift_reg_in[M-2:0], MOSI};
```

MoSAIC compatible `ODIN_MOSAIC_CTRL.sv` logic

```
always @(*) begin
|   case (ODINAddr[17:16])
|       ODIN_OP_NEUR: ODIN_data_out_mux = NEUR_STATE;
|       ODIN_OP_SYN: ODIN_data_out_mux = SYNARRAY_RDATA;
|       ODIN_OP_CTRL: begin
|           case (ODINAddr[15:0])
|               GATE_ACTIVITY_ENABLE: ODIN_data_out_mux = {31'b0, GATE_ACTIVITY};
|               OPEN_LOOP_ENABLE:     ODIN_data_out_mux = {31'b0, OPEN_LOOP};
|               A_S_C_NN_ENABLE:      ODIN_data_out_mux = {31'b0, AER_SRC_CTRL_nNEUR};
|               MAX_NEUR_ENABLE:      ODIN_data_out_mux = {{(32-M){1'b0}}, MAX_NEUR};
|               default:              ODIN_data_out_mux = 32'h0;
|           endcase
|       end
|       default: ODIN_data_out_mux = 32'h0;
|   endcase
end

assign mem_rdata_axi = ODIN_data_out_mux;
```

controller.v finite state machine (FSM) changes

The FSM in controller.v no longer stays in the WAIT_SPIDN state, and goes straight to the WAIT state. This was done because the FSM would often get stuck in the WAIT_SPIDN state since data is constantly being fed into the controller through the SPI hex files, which is not what was previously expected by the FSM. The state machine now works by staying in a certain state (during neuron programming, this would be W_NEUR) until a new address is read.

Connecting TinyODIN to the NoC through acc_modin.sv

TinyODIN's AER interface block has appropriate signals that handle input and output, denoted by AERIN and AEROUT.

```
tinyMODIN #(
    .N      (N),
    .M      (M)
) tinyMODIN_inst (
    .CLK      (clk_ctrl),
    .RST      (clk_ctrl_rst_high),
    .mem_valid_axi      (mem_valid_axi),
    .mem_addr_axi      (mem_addr_axi),
    .mem_wdata_axi      (mem_wdata_axi),
    .mem_wstrb_axi      (mem_wstrb_axi),
    .mem_rdata_axi      (mem_rdata_axi),
    .rvControl      (rvControl),

    // message passing interface!
    .AERIN_ADDR      (mm_mem_wdata[9:0]),
    .AERIN_REQ(mm_mem_valid & mm_mem_wstrb),
    .AERIN_ACK      (), // disconnected

    .AEROUT_ADDR      (AEROUT_ADDR),
    .AEROUT_REQ      (AEROUT_REQ),
    .AEROUT_ACK      (AEROUT_ACK),
    .SCHED_FULL      ()
);

mem_spy_modin#(
    .NOC_BUFFER_ADDR_W(NOC_BUFFER_ADDR_W),
    .XY_SZ(XY_SZ),
    .OFFSET_SZ(OFFSET_SZ)
) mem_spy_modin(
    .clk_ctrl      (clk_ctrl),
    .clk_ctrl_rst_low      (clk_ctrl_rst_low),
    .clk_ctrl_rst_high      (clk_ctrl_rst_high),
    .clk_line      (clk_line),
    .clk_line_rst_low      (clk_line_rst_low),
    .HsrcId(HsrcId),

    .mem_ready_rv(AEROUT_ACK),
    .mem_addr_rv(scratchpad_write_addr),
    .mem_wdata_rv({24'b0, AEROUT_ADDR}),
    .mem_wstrb_rv(AEROUT_REQ),
    .mem_valid_rv(AEROUT_REQ),

    .unblock(1'b1),
    .spy_idle(),
    .local_mem(1'b0),

    .stream_out_TREADY      (stream_out_TREADY),
    .stream_out_TVALID      (stream_out_TVALID),
    .stream_out_TDATA      (stream_out_TDATA),
    .stream_out_TKEEP      (stream_out_TKEEP),
    .stream_out_TLAST      (stream_out_TLAST)
);
```

The AERIN signals from MoDIN connect to other signals denoted by mm_mem from the NoC Decoder block.

The AEROUT signals connect to the picorv signals in the output NoC buffer. An interface exists between the accelerator and the output NoC buffer, the mem_spy_modin, which we use to spy on the signal transactions between the processor and memory. Inside this block we control when packets from MoDIN get sent to a scratchpad tile with a FSM.

Tutorial

Steps to run a MoSAIC simulation with MoDIN:

Step 1: Set up the tile array with the perl script.

1. Add the MoDIN accelerator to your perl script. `tools/generate/modin_mosaic.pl`

```
#- Create structure to hold new types of tile (Needed)
%new_tile;

#- Pair the verilog file with an alias for each new tile (User)
$new_tile{'modin'} = 'Tile_modin';

#- Add it to the parameters (Needed)
$param{'new_tile'} = \%new_tile;
```

2. Set tile array parameters and specify the C file (we are using pico_snn.c). Here's an example of a 2x2 array with 1 Picorv32, 2 scratchpads, and 1 MoDIN accelerator.

```
#- 2x2 Tile array
$path = "$ENV{PWD}";
$fw_path = "$path/../../picorv_c/c_modin/";
$param{'r'} = 2;
$param{'c'} = 2;
$c_file = "pico_snn";

@tile_array = ([ 'pico', 'spad'],
               [ 'spad', 'modin']);

$param{'firmware_path'} = $fw_path;
```

3. Once the perl script is executed, it will generate hex files for the pico. For each pico tile used, put the hex file from the corresponding tile id in the @pico_program array.

```
@pico_program = ("${c_file}32_0.hex", '',
                 # '', '');
                 |'', 'hex_files/SPI_file_closedLoopOnly.hex');
                 # '', 'hex_files/SPI_OpenLoopPhase1.hex');
                 # '', 'hex_files/SPI_OpenLoopPhase2.hex');
```

MoDIN uses a hex files for SPI signal configuration generated by the TinyODIN testbench. (`tbench.sv`). The hex files perform neuron and synapse programming with pre-defined test weights, and one of three experiments, closed loop, open loop phase 1, or open loop phase 2 (See TinyODIN documentation for more detail on these experiments).

Step 2: Modify the C code. `tools/picorv_c/c_modin/ pico_snn.c`

MoSAIC uses the `mPut()` C function to send messages to tiles based on their tile id. The tile id for each accelerator is available in the generated file, `mosaic_defines.h` after running the perl script you wrote. In this particular 2x2 MoSAIC tile array, MoDIN is tile 9, and `modin_tile_num` should be set equal to 9, bit shifted by 12.

```
uint32_t data;
uint32_t addr = 9;
addr = (addr << 12);

// CLOSED LOOP
if (DO_CLOSED_LOOP){
    data = 0x253; // Virtual value 5 event to neuron 3
    // phase 2
    if (phase){
        for (j=0; j<2; j=j+1){
            mPut(data,addr);
        }
    }
}
```

Using the `mPut()`, AER inputs may be sent to MoDIN. Using `mGet()`, data can be retrieved from the scratchpad.

Notes

Any tile array with a MoDIN accelerator will need a pico for instructions and a scratchpad (SPAD) to store AEROUT sends. In our example we send AEROUT data to the SPAD on tile 8, change accordingly for your set up. `src/Tile_HDL/modin_tile/ acc_modin.sv`

```
logic [31:0] tile_addr;
assign tile_addr = 8 << 12;
assign scratchpad_write_addr = tile_addr | scratchpad_counter;
```

You can create a new vivado project by running the perl script with the parameter settings below. Afterwards, set `vivado_project` to 0 so that your simulation configurations (signal colors, groups, etc) won't be overwritten when you re-run the script. `tools/generate/ modin_mosaic.pl`

```
#- Running with Vivado
$param{'vivado'} = 1;
$param{'vivado_project'} = 1;
$param{'run_sim'} = 0;
$param{'sim_only'} = 0;
```

Impact on Laboratory and National Missions

MoSAIC now has neuromorphic capabilities as a result of the MoDIN project. Applications for SNN processors exist in Radiation Sensors, 5G Edge Sensors, High Energy Physics, Xray Sensors, Compression, and AI. This work is of particular interest to areas in the nuclear sciences. Project 38 is funded by the Department of Defense (DOD), the DOE Office of Science, and the National Nuclear Security Administration (NNSA).

Conclusion

Features that exist in the ODIN chip will also be integrated before testing on an FPGA, since training of the SNN is much easier with on-chip learning capabilities. Other SNNs will also exist on MoSAIC in the future, currently work is being done in a similar project to implement the SNE SNN on MoSAIC. Exploration of neuromorphic architectures can bring solutions to important problems that exist in computing today, and MoDIN will offer important new insight to these issues.

Acknowledgments

This work was supported in part by the U.S. Department of Energy, Office of Science, Office of Workforce Development for Teachers and Scientists (WDTS) under the Community College Internship (CCI) program.

References

- [1] Shalf J. 2020 “The future of computing beyond Moore’s Law.” Phil. Trans. R. Soc. A 378:20190061. <http://dx.doi.org/10.1098/rsta.2019.0061>
- [2] Brader JM, Senn W, Fusi S. Learning real-world stimuli in a neural network with spike-driven synaptic dynamics. Neural Comput. 2007 Nov;19(11):2881-912. doi: 10.1162/neco.2007.19.11.2881. PMID: 17883345.
- [3] C. Frenkel, M. Lefebvre, J.-D. Legat and D. Bol, ”A 0.086-mm² 12.7-pJ/SOP 64k-Synapse 256-Neuron Online-Learning Digital Spiking Neuromorphic Processor in 28-nm CMOS,” IEEE Transactions on Biomedical Circuits and Systems, vol. 13, no. 1, pp. 145-158, 2019.

Appendix

Notable outcomes

Notable outcomes from this appointment include presentations of this work at the Computing Sciences Area and the WD&E poster sessions, on August 6th and August 7th respectively. I (Laura Kallem) also participated in a science communication competition, Ignite Off, on July 14th.

Prior to this experience I had no experience with hardware design, and likely wouldn't have had an opportunity to explore Computer Architecture until I began studying at a 4-year university. The CCI program was an invaluable learning opportunity and allowed me to explore a potential career in research, which I will surely continue working towards. I will work on this project as a volunteer Affiliate until it is finished, and hope to continue doing similar work in future internships.

Participants

Laura Kallem: Researched and compiled information about open source SNNs, involved in every step of integration with the goal of learning about and understanding computer architecture.

Mentors

Farzad Fatollahi-Fard: Mentored and taught Laura methods of debugging digital hardware systems, offered Laura extensive support navigating MoSAIC.

Patricia Gonzalez-Guerrero (PI): Creator of MoSAIC, met weekly with Laura to offer mentorship and support with TinyODIN integration.